Binary Search Drill Session

Codeforces

> 702C Cellular Network
>
> ### 702C Cellular Network
>
> **Problem Link** : [702C Cellular Network]
>
> **Difficulty** : #Difficulty-Cf-1500
>
> **Platform** : #Platform-Codeforces
>
> ---
>
> ### Solution
>
> **Approach - 1 (Binary Search)**

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

// Can we provide each city by atleast 1 cellular tower within radius <= X ?
// Check that all n points (c1, ... cn) are covered in m ranges ([t1-X, t1+X],
... [tn-X, tn+X]) ?
bool check(int X, set<int> c, vector<int>& t) {    // O(mlogn + n)
    for (auto cur_t:t) {
        int st_range = cur_t-X, end_range = cur_t+X;

        // Delete all cities present in range of current tower
        auto it = c.lower_bound(st_range);
        while(it != c.end() && *it <= end_range) {
            it = c.erase(it);    // Erase returns the next iterator
        }
    }
    return c.empty();
}

void solve() {   // O(n + m + ((logn) x (mlogn + n)))
    int n, m;
    cin >> n >> m;

    set<int> c;
    vector<int> t(m);
    for(int i=0; i<n; i++) {
        int city;
        cin >> city;
        c.insert(city);
    }
```

```
    for(int i=0; i<m; i++) cin >> t[i];

    int ans=0;
    int lo=0;
    int hi=max(abs(*c.rbegin() - *t.begin()), abs(*c.begin() - *t.rbegin()));
    while(lo <= hi) {
        int mid = lo + ((hi-lo) >> 1);
        if(check(mid, c, t)) {
            ans = mid;
            hi = mid-1;
        } else lo = mid+1;
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

Approach - 2 (Observation)

As you increase the radius, there will be a last point (i.e. there would be a limiting city which when touched covers everything). Every city requires it to have 1 of the towers.

For every city you can determine the closest tower to it, and that tower will decide the coverage of that particular city. So for all cities you get the limiting radius required for coverage of each city, now answer would be maximum of all limiting radius (of all cities) bcoz city with highest limiting radius needs to get covered as well (bcoz it's the limiting city)

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int n, m;
    cin >> n >> m;

    vector<int> c(n), t(m);
    for(int i=0; i<n; i++) cin >> c[i];
    for(int i=0; i<m; i++) cin >> t[i];

    int ans=0;
    for(auto cur_c:c) {
        auto it = lower_bound(t.begin(), t.end(), cur_c);
        int closest_td = INT_MAX;    // Closest tower distance (Limiting radius)

        if(it != t.end()) {
            int next_td = *it - cur_c;
            closest_td = min(closest_td, next_td);
        }
        if(it != t.begin()) {
            it--;
            int prev_td = cur_c - *it;
            closest_td = min(closest_td, prev_td);
        }
```

```
        ans = max(ans, closest_td);
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

CFFTD Breakdown

- **C (Concept/topic used in the problem)**
  Approach - 1 : Binary Search
  Approach - 2 : Observation, STL

- **F (Framework, questions that would lead to new observations/forms)**
  Approach - 1 : How will you check each tower covers how many cities ?
  Even if you're able to check that, then what data structure would you use to remove cities that are covered based on current value of mid ?
  Approach - 2 : For each city, can we say that it has a minimum requirement of radius (limiting) based on towers on it's left and right ? If yes, how would you obtain this limiting radius ?

- **F (Form, which style of problem, as discussed in various topics)**
  Approach - 1 : Binary Search on Answer

- **T (Tactics, what misc idea was used to reduce the time complexity/constraint)**
  Approach - 1 : Check function reduces to, Check that all n points (c1, ... cn) are covered in m ranges ([t1-X, t1+X], ... [tn-X, tn+X]) ?
  For implementing this check function, store cities in a set
  Approach - 2 : Consider each city as limiting city, and find it's limiting radius, then answer would be maximum of limiting radius's of all cities

- **D (Debug issues faced, what to keep in mind next time)**
  Approach - 1 : `hi` would be max of distance of farthest tower from closest city and closest tower from farthest city.
  Keep in mind that `set.erase(it)` returns the next iterator

251A Points on Line

251A Points on Line

Problem Link : 251A Points on Line

Difficulty : #Difficulty-Cf-1300

Platform : #Platform-Codeforces

Solution

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int n, d;
    cin >> n >> d;

    vector<int> a(n);
    for(int i=0; i<n; i++) cin >> a[i];

    // Edge case
    if(n < 3) {
        cout << "0\n";
        return;
    }

    int ans=0;
    for(int st=0; st<=n-3; st++) {
        auto idx = upper_bound(a.begin(), a.end(), d+a[st]) - a.begin();
        int avail_ele = idx - (st + 1);
        if(avail_ele < 2) continue;    // We need atleast 2 elements except start
        int nC2 = ((avail_ele)*(avail_ele-1)) / 2;
        ans += nC2;
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

CFFTD Breakdown

- **C (Concept/topic used in the problem)**
  Binary Search, Combinatorics

- **F (Framework, questions that would lead to new observations/forms)**
  For counting, what framework of BS do we use ?
  Every element once being the smallest element, how many sets can we create ?
  For each start, how can we decide what's last element till which we can choose remaining 2 elements ?
  What will you use to calculate ways in which you can choose 2 elements from available

elements ?

- **F (Form, which style of problem, as discussed in various topics)**
  Since we're talking about counting, so it's form would be Binary Search on Every Start

- **T (Tactics, what misc idea was used to reduce the time complexity/constraint)**
  Considering start as the smallest element among all 3 that we've to chose
  For each start, we can choose remaining 2 elements upto a point where distance $\leq d$. So available number of choices for selecting remaining 2 points are given by number of elements $\leq d$ which are ahead of current start, it's given by `ub_ind - (st + 1)`
  To calculate number of ways of choosing 2 elements out of available elements, use Combination
  Our answer could be expressed as :

$$\sum_{st=0}^{n-3} {}^{ub-(st+1)}\mathrm{C}_2$$

  where,
  `ub` denotes the upper bound of `d + a[st]` (as it gives number of elements $\leq d$)

- **D (Debug issues faced, what to keep in mind next time)**
  If available elements ≤ 2, then number of ways of selecting 2 elements is 0

AtCoder

143D Triangles

143D Triangle

Problem Link : 143D Triangles

Difficulty : #Difficulty-AC-400

Platform : #Platform-AtCoder

Solution

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int n;
    cin >> n;

    vector<int> L(n);
    for(int i=0; i<n; i++) cin >> L[i];
    sort(L.begin(), L.end());
```

```
    int ans=0;
    for(int a=0; a<n; a++) {
        for(int b=a+1; b<n; b++) {
            int ub_idx = lower_bound(L.begin(), L.end(), L[a] + L[b]) -
L.begin();
            int cnt_c = ub_idx - (b+1);    // No. of ways of choosing c (for
current chosen a & b)
            ans += cnt_c;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

CFFTD Breakdown

- **C (Concept/topic used in the problem)**
  Binary Search, Combinatorics

- **F (Framework, questions that would lead to new observations/forms)**
  If you choose `a`, `b`, `c` such that $a < b < c$, then among given 3 conditions, how many conditions you need to satisfy explicitly ?

- **F (Form, which style of problem, as discussed in various topics)**
  Since we're talking about counting, so it's form would be Binary Search on Every Start

- **T (Tactics, what misc idea was used to reduce the time complexity/constraint)**
  If $a < b < c$, then we only need to satisfy 1 single condition explicitly i.e. $c < a + b$
  Since we assumed `c` as largest side, so the conditions $a < b + c$ and $b < a + c$ hold true by triviality
  So for each possible combination of `a` and `b` ($a < b$), we count total number of possibilities of `c` (based on the condition $c < a + b$) using Combination, then add it up to obtain the answer

- **D (Debug issues faced, what to keep in mind next time)**

144D Water Bottle

144D Water Bottle

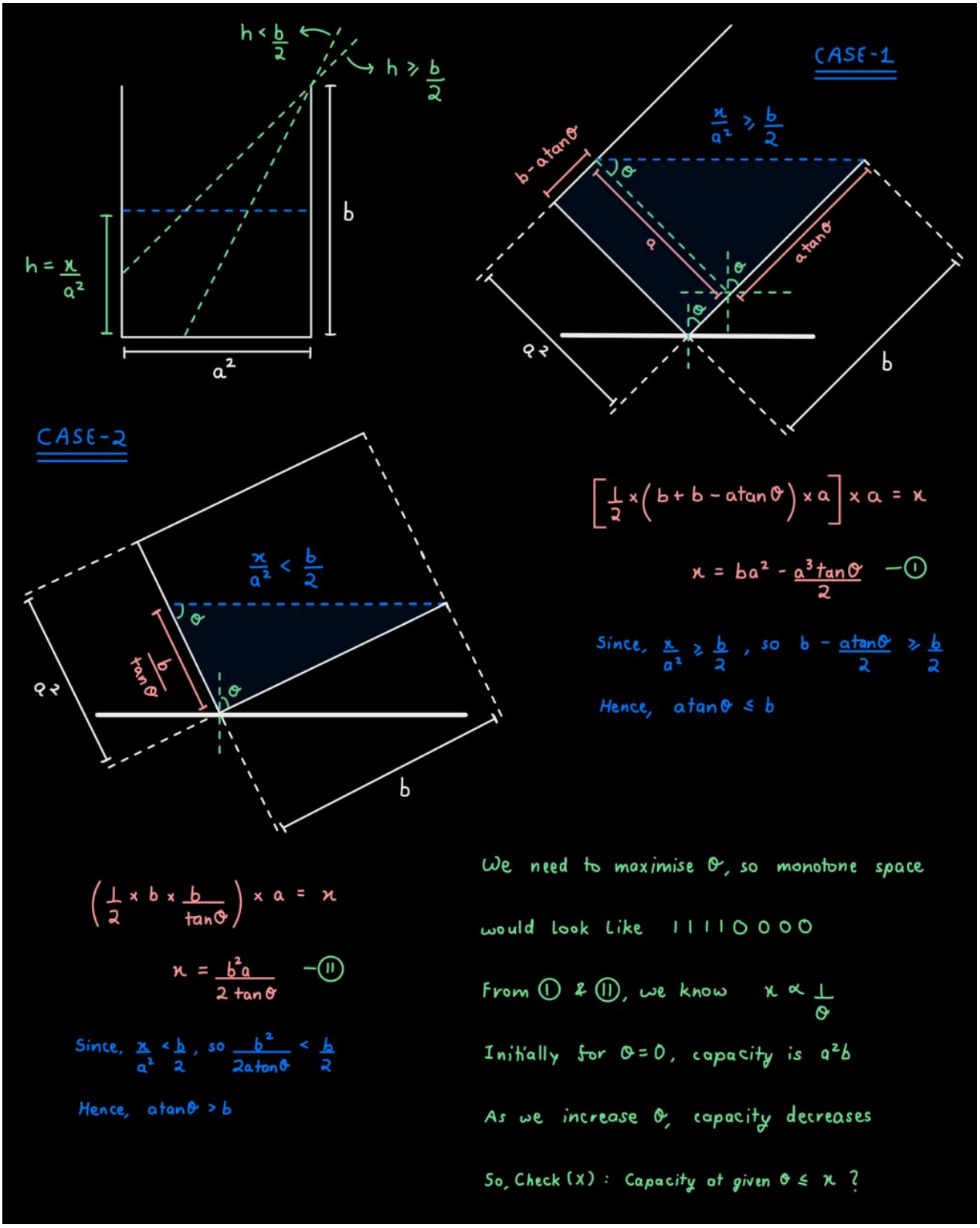
Problem Link : 144D Water Bottle

Difficulty : #Difficulty-AC-400

Platform : #Platform-AtCoder

Solution

Dry Run



Code

```
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define ld long double
```

```
ld PI = 4.0 * atan(1.0);    // atan(1.0) = pi/4
ld EPS = 1e-12;

// Does water stay intact at given angle theta ?
// Capacity at given theta <= X ?
bool check(ld theta, int x, int a, int b) {
    ld capacity;

    // Case-1 : height >= b/2
    if(a * tan(theta) <= b) {
        capacity = (b * a * a) - ((a * a * a * tan(theta))/2.0);
    }
    // Case-2 : height < b/2
    else {
        capacity = (b * b * a) / (2 * tan(theta));
    }

    return (capacity >= x);
}

void solve() {
    int a, b, x;
    cin >> a >> b >> x;

    ld ans=0.0;
    ld lo=0.0, hi=PI/2;
    while(abs(hi - lo) > EPS) {
        ld mid = (lo + hi) / 2.0;
        if(check(mid, x, a, b)) {
            ans = mid;
            lo = mid;
        } else hi = mid;
    }
    cout << fixed << setprecision(10) << ans / PI * 180 << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

CFFTD Breakdown

- **C (Concept/topic used in the problem)**
  Binary Search

- **F (Framework, questions that would lead to new observations/forms)**
  What happens when height of water `< b/2` and `>= b/2` ? (Observe by drawing each)

- **F (Form, which style of problem, as discussed in various topics)**
  Binary Search on Answer, Binary Search on Real Domain

- **T (Tactics, what misc idea was used to reduce the time complexity/constraint)**
  Check(theta) : Does water stay intact at given angle theta ?
  Check(theta) : Capacity at given theta <= X ?
  While implementing Check function, deal with both cases of height separately

- **D (Debug issues faced, what to keep in mind next time)**
  `tan` takes input in radians
  We need to print answer in degrees
  Choose `EPS` value carefully according to precision required

146C Buy an Integer

146C Buy an Integer

Problem Link : [146C Buy an Integer]

Difficulty : #Difficulty-AC-300

Platform : #Platform-AtCoder

Solution

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

int cnt_digits(int num) {
    int cnt=0;
    while(num) {
        cnt++;
        num /= 10;
    }
    return cnt;
}

// Can we buy integer X with bl as budget limit ?
bool check(int X, int bl, int A, int B) {
    int cnt_digits_X = cnt_digits(X);
    int cost_X = (A * X) + (B * cnt_digits_X);
    return cost_X <= bl;
}

void solve() {
    int a, b, x;
    cin >> a >> b >> x;

    int ans=0;
    int lo=1, hi=1e9;
    while(lo <= hi) {
```

```
        int mid = lo + ((hi-lo) >> 1);
        if(check(mid, x, a, b)) {
            ans = mid;
            lo = mid+1;
        } else hi = mid-1;
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

CFFTD Breakdown

- **C (Concept/topic used in the problem)**
  Binary Search

- **F (Framework, questions that would lead to new observations/forms)**
  Can you say something about monotonicity of given function to calculate cost ?

- **F (Form, which style of problem, as discussed in various topics)**
  Binary Search on Answer

- **T (Tactics, what misc idea was used to reduce the time complexity/constraint)**
  The given function to calculate cost is monotonically increasing in nature
  Check function says that Can we buy integer `mid` with `x` as the budget limit ?

- **D (Debug issues faced, what to keep in mind next time)**
  If `hi` wasn't explicitly mentioned to be `1e9`, then what `hi` would you take ?
  Assume $B \cdot d(N)$ is not present, then $A \cdot N \leq X$
  So,

  $$N \leq \frac{X}{A}$$

  Hence, `hi = X/A`

155D Pairs

155D Pairs

Problem Link : 155D Pairs

Difficulty : #Difficulty-AC-400

Platform : #Platform-AtCoder

Solution

```
#include<bits/stdc++.h>
using namespace std;
#define int long long

// Can we make product <= X, with number of pairs >= k ?
// (# of pairs whose product <= X) >= k
bool check(int X, int k, vector<int> pos, vector<int> neg) {
    int cnt_pairs=0;

    // Count no. of possible pairs of form (-ve, +ve) with product <= X
    for(int st=0; st<pos.size(); st++) {
        int ans=-1;
        int lo=0, hi=neg.size()-1;
        while(lo <= hi) {
            int mid = lo + ((hi-lo) >> 1);
            if(pos[st] * neg[mid] <= X) {
                ans = mid;
                lo = mid+1;
            } else hi = mid-1;
        }
        cnt_pairs += ans - 0 + 1;
    }

    // For negative product, no form of pair except (-ve, +ve) is valid
    if(X < 0) return (cnt_pairs >= k);

    // Count no. of possible pairs of form (+ve, +ve) with product <= X
    for(int st=0; st<pos.size(); st++) {
        int ans=st;
        int lo=st+1, hi=pos.size()-1;
        while(lo <= hi) {
            int mid = lo + ((hi-lo) >> 1);
            if(pos[st] * pos[mid] <= X) {
                ans = mid;
                lo = mid+1;
            } else hi = mid-1;
        }
        cnt_pairs += ans - (st+1) + 1;
    }

    // Count no. of possible pairs of form (-ve, -ve) with product <= X
    for(int st=0; st<neg.size(); st++) {
        int ans=neg.size();
        int lo=st+1, hi=neg.size()-1;
        while(lo <= hi) {
            int mid = lo + ((hi-lo) >> 1);
            if(neg[st] * neg[mid] <= X) {
                ans = mid;
                hi = mid-1;
            } else lo = mid+1;
        }
        cnt_pairs += (neg.size()-1) - (ans) + 1;
```

```
    }

    return (cnt_pairs >= k);
}

void solve() {
    int n, k;
    cin >> n >> k;

    vector<int> pos, neg;
    for(int i=0; i<n; i++) {
        int x;
        cin >> x;
        if(x >= 0) pos.push_back(x);
        else neg.push_back(x);
    }
    sort(pos.begin(), pos.end());
    sort(neg.begin(), neg.end());

    int ans=-1e18;
    int lo=-1e18, hi=1e18;
    while(lo <= hi) {
        int mid = lo + ((hi-lo) >> 1);
        if(check(mid, k, pos, neg)) {
            ans = mid;
            hi = mid-1;
        } else lo = mid+1;
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);
    solve();
}
```

CFFTD Breakdown

- **C (Concept/topic used in the problem)**
  Binary Search

- **F (Framework, questions that would lead to new observations/forms)**
  How can you handle negative products ?
  How many variation of pairs can you get based on selecting `-ve` and `+ve` numbers ?

- **F (Form, which style of problem, as discussed in various topics)**
  Binary Search on Answer, Binary Search on Every Start

- **T (Tactics, what misc idea was used to reduce the time complexity/constraint)**
  3 variation of pairs can be obtained, i.e. `(+ve, +ve)`, `(-ve, -ve)` and `(-ve, +ve)`
  Check(X) : Can we make product <= X, with number of pairs >= k ?

For implementing this check function, separately consider each variation of pair and use BS on Every Start to count number of pairs of each type

- **D (Debug issues faced, what to keep in mind next time)**
  Keep in mind products can be negative as well
  In form BS on Every Start, search space starts from `st + 1`